SYSTÈME MULTI-AGENTS — INTELLIGENCE ARTIFICIELLE

Autonomous Research Agent

ARA — Version 2.0

Un système qui lit et synthétise des centaines d'articles scientifiques à votre place — en moins de 15 minutes.

Bilel Ammouri | Avril 2025

Plan de la Présentation

1. Contexte et Problématique
2. Architecture du Système
3. Les 6 Agents en Détail
4. Configuration et Utilisation
5. Résilience et Gestion des Erreurs
6. Tableau de Bord des Fonctions
7. Recommandations et Conclusion

Le Problème Que Nous Résolvons

Sans ARA — La réalité d'aujourd'hui

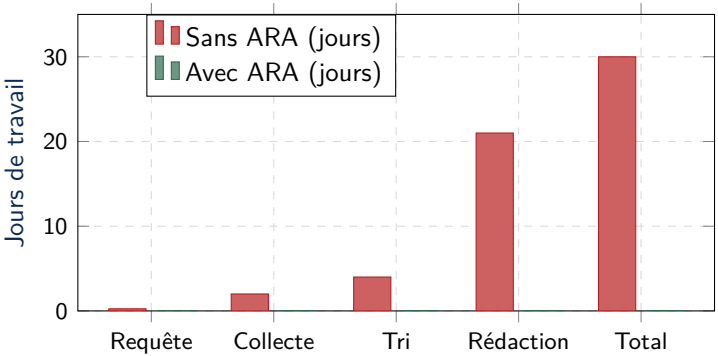
- 3+ millions d'articles publiés/an
- Tri manuel : 3 à 6 semaines de travail
- Risque d'oublier des sources clés
- Coût humain élevé
- Résultat non-reproductible

Avec ARA — La nouvelle réalité

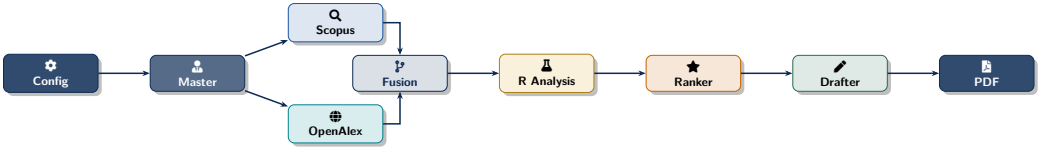
- Même travail effectué en **15 minutes**
- Couverture de 50+ articles simultanément
- 2 bases mondiales interrogées
- Reproductible à l'identique
- Rapport PDF prêt à l'emploi

ARA transforme 3 semaines de travail d'expert en 15 minutes automatisées. Gain $\times$1000.

Les Chiffres du Gain



Architecture — La Grande Image



Master Agent

Orchestre tout

Collecteurs

Scopus + OpenAlex

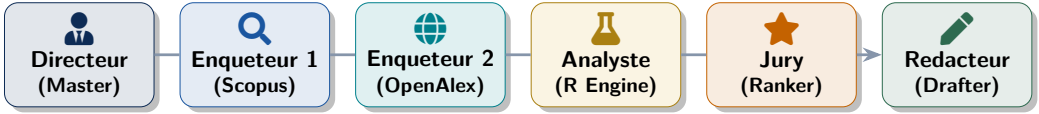
Analyseur + Ranker

R + Scoring Python

Drafter IA

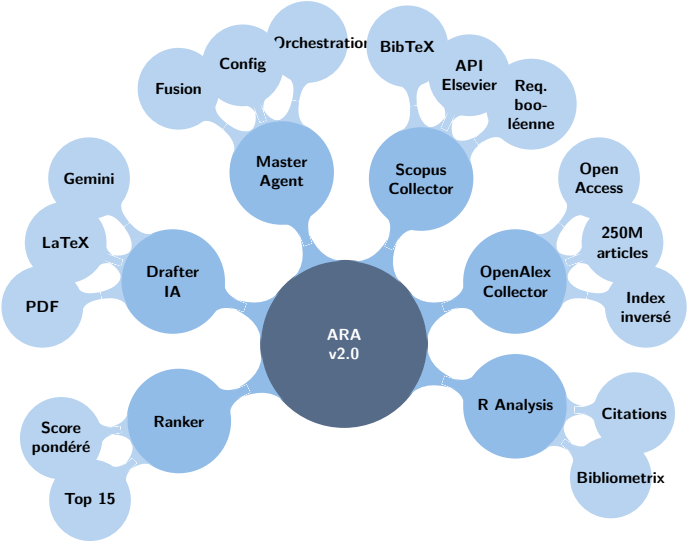
Gemini + LaTeX

L'Analogie de l'Équipe de Recherche



Chaque role est tenu par un **programme specialise**. Ils se passent le travail en sequence, comme une **chaine de montage intelligente**.

Mindmap Globale du Système



Agent 1 — Master Agent

Identité

Fichier	`master_agent.py`
Rôle	Chef de projet
Langage	Python 3

Fonctions clés

- `find_r_executable()`
- `run_step()`
- `merge_results()`
- `main()`

Séquence d'exécution

1. Vérifier la clé Gemini
2. Choisir la source (Scopus/OpenAlex/Les 2)
3. Lancer `scopus_collector.py`
4. Lancer `openalex_collector.py`
5. Fusionner + Dédupliquer
6. Lancer `analysis_engine.R`
7. Lancer `paper_ranker.py`
8. Lancer `review_drafter.py`
9. Confirmer : *"MISSION SUCCESSFUL"*

Agent 1 — Fonction `run_step()`

INPUT

name (str) : Nom de l'étape | **command** (list) : Commande à exécuter

```python
def run_step(name, command):
    if command[0] == "Rscript":
        r_path = find_r_executable()
        if r_path:
            command[0] = r_path  # remplace "Rscript" par le vrai chemin
        else:
            return False         # R non installe
    try:
        subprocess.run(command, check=True)
        return True              # succes
    except subprocess.CalledProcessError:
        return False             # echec
```

OUTPUT

bool : `True` si l'étape réussit, `False` sinon

Agent 1 — Fonction `merge_results()`

INPUT

sources (list) : ex. `["scopus", "openalex"]`

```python
def merge_results(sources):
    dfs = []  # liste de tableaux a fusionner
    if "scopus" in sources:
        dfs.append(pd.read_csv("scopus_metadata.csv"))
    if "openalex" in sources:
        dfs.append(pd.read_csv("openalex_metadata.csv"))
    combined = pd.concat(dfs, ignore_index=True)
    # Deduplication par DOI puis par titre
    combined = combined.drop_duplicates(subset=["dc:title"])
    combined.to_csv("collected_metadata.csv", index=False)
```

OUTPUT

Fichier : `collected_metadata.csv` — tableau unique sans doublons

Agent 2 — Scopus Collector

Identité

Fichier	`scopus_collector.py`
Rôle	Bibliothécaire Elsevier
API	api.elsevier.com

Fonctions

```
get_config()
suggest_boolean_query()
fetch_scopus_data()
process_entries()
export_to_bibtex()
```

Données collectées

- Titre (`dc:title`)
- Auteur (`dc:creator`)
- Année (`prism:coverDate`)
- Citations (`citedby-count`)
- Résumé (`dc:description`)
- DOI (`prism:doi`)
- Mots-clés (`authkeywords`)
- Journal (`prism:publicationName`)

Agent 2 — Fonction suggest_boolean_query()

INPUT

client (genai.Client), **model_id** (str), **theme** (str)
Exemple de thème : *"équité intergénérationnelle et fiscalité carbone"*

```python
def suggest_boolean_query(client, model_id, theme):
    prompt = f"""Generate a Scopus boolean query for: "{theme}".
    Use TITLE-ABS-KEY format. Return ONLY the query."""

    response = client.models.generate_content(
        model=model_id, contents=prompt
    )
    query = response.text.strip().replace("`", "")
    if "TITLE-ABS-KEY" not in query:
        query = f"TITLE-ABS-KEY({query})"
    return query
```

OUTPUT

str : Requête optimisée, ex : `TITLE-ABS-KEY("intergenerational equity" AND "carbon tax")`

Agent 2 — Fonction `fetch_scopus_data()`

INPUT

query (str), **api_key** (str), **limit** (int=50)

```python
def fetch_scopus_data(query, api_key, limit=50):
    url = "https://api.elsevier.com/content/search/scopus"
    headers = {"X-ELS-APIKey": api_key,
               "Accept": "application/json"}
    params = {"query": query, "count": min(limit, 25),
              "field": "dc:title,dc:creator,..."}
    resp = requests.get(url, headers=headers,
                        params=params, timeout=20)
    if resp.status_code == 200:
        return resp.json()["search-results"]["entry"]
    return []
```

OUTPUT

list[dict] : Liste de dictionnaires (1 dict = 1 article avec ses métadonnées)

Agent 3 — OpenAlex Collector

Particularité unique : l'Index Inversé

OpenAlex stocke les résumés de façon compressée.

"Climate"→[0]
"change"→[1]
"policy"→[2,5]

reconstruct →

"Climate change policy [...] policy"

Avantages vs Scopus

- **250M+ articles** (vs 90M Scopus)
- **Gratuit** (Open Access)
- Articles récents bien couverts
- Données de citations publiques

Sorties

- `openalex_metadata.csv`
- `openalex_results.bib`

Agent 3 — Fonction reconstruct_abstract()

INPUT

inverted_index (dict) : ex. `{"Climate":[0], "change":[1], "poses":[2]}`

```python
def reconstruct_abstract(inverted_index):
    if not inverted_index:
        return ""
    word_index = {}
    # Inverser le dictionnaire : mot->pos devient pos->mot
    for word, locations in inverted_index.items():
        for loc in locations:
            word_index[loc] = word
    # Reconstruire dans l'ordre des positions
    highest = max(word_index.keys())
    parts = [word_index.get(i,"") for i in range(highest+1)]
    return " ".join(parts)
```

OUTPUT

str : Résumé complet reconstruit en texte lisible

Agent 4 — Analysis Engine (R)

Rôle

Analyser scientifiquement l'influence des articles collectés

Métriques calculées

TC	Total Citations
TI	Titre de l'article
AU	Auteurs
PY	Année de publication
DI	DOI

Librairie : Bibliometrix

Standard mondial pour l'analyse bibliométrique. Utilisé par des milliers de chercheurs.

Entrée / Sortie

IN : `results.bib`
OUT : `citation_metrics.csv`
OUT : `bibliometric_summary.png`

Note

Cet agent est optionnel. Si R n'est pas installé, le pipeline continue.

Agent 5 — Paper Ranker

La formule de scoring

$$\text{Score} = \text{Cit} \times 2 + (\text{An} - 2000)$$

Pourquoi $\times 2$ pour les citations ?
La citation par d'autres chercheurs est le **meilleur indicateur objectif** de l'impact scientifique. La récence compte moitié moins.

Exemple de calcul

Article	Cit.	Année	Score
A	200	2018	**418**
B	50	2024	**124**
C	10	2024	**44**

Sortie

`ranked_articles.json`
Top 15 articles classés avec leurs métadonnées complètes

Agent 5 — Fonction rank_papers()

INPUT

Aucun paramètre direct — lit `collected_metadata.csv` + `citation_metrics.csv`

```python
def rank_papers():
    df = pd.read_csv("collected_metadata.csv")
    df['citations_count'] = pd.to_numeric(
        df['citations'], errors='coerce').fillna(0)
    df['year_val'] = df['prism:coverDate'].str[:4].astype(int)
    # FORMULE DE SCORING
    df['rank_score'] = df['citations_count']*2 + (df['year_val']-2000)
    top15 = df.sort_values('rank_score', ascending=False).head(15)
    json.dump(top15.to_dict('records'), open("ranked_articles.json","w"))
```

OUTPUT

Fichier : `ranked_articles.json` — Top 15 sélectionnés avec score

Agent 6 — Review Drafter

Identité

Fichier	`review_drafter.py`
IA	Gemini (Google)
Sortie	`main.tex` + `main.pdf`

8 fonctions clés

`load_config()` · `load_data()`
`latex_escape()` · `initialize_client()`
`call_with_retry()` · `generate_dynamic_structure()`
`draft_section()` · `create_latex_master()`
`run_compilation()`

Séquence de rédaction

1. Charger les 15 meilleurs articles
2. Demander à Gemini de proposer 4–6 sections thématiques
3. Rédiger l'introduction (200 mots)
4. Rédiger chaque section (700 mots)
5. Assembler le document LaTeX
6. Compiler le PDF (3 passes)

Agent 6 — Fonction call_with_retry()

INPUT

client, **model_id**, **prompt** (str), **max_retries** (int=5)

```python
def call_with_retry(client, model_id, prompt, max_retries=5):
    for attempt in range(max_retries):
        try:
            response = client.models.generate_content(
                model=model_id, contents=prompt)
            return response.text.strip()
        except Exception as e:
            if "429" in str(e) or "RESOURCE_EXHAUSTED" in str(e):
                wait = (2**attempt) + random.random() + 5
                # 6s, 7s, 9s, 13s, 21s (backoff exponentiel)
                time.sleep(wait)
            else:
                return None
    return None
```

OUTPUT

str : Texte généré | **None** : si toutes les tentatives échouent

Agent 6 — Fonction draft_section()

INPUT

client, **model_id**, **theme** (str), **section_title** (str), **articles** (list), **target_words** (int)

```python
def draft_section(client, model_id, theme,
                  section_title, articles, target_words):
    context = ""
    for i, a in enumerate(articles[:15]):
        context += f"ARTICLE {i+1}: {a['title']} ({a['year']})\n"
        context += f"Abstract: {a.get('abstract','')[:1200]}\n\n"

    prompt = f"""Write a ~{target_words}-word academic synthesis
    for section "{section_title}" of a review on "{theme}".
    Use \\citep{{idX}} for citations. Temperature=0.3."""

    cfg = types.GenerateContentConfig(temperature=0.3)
    return call_with_retry(client, model_id, prompt, cfg)
```

OUTPUT

str : Corps LaTeX de la section (700 mots avec citations)

Agent 6 — Fonction `latex_escape()`

INPUT

text (str) : Texte brut généré par Gemini (peut contenir %, $, &, _, etc.)

```python
def latex_escape(text):
    chars = {"&":r"\&", "%":r"\%", "$":r"\$",
             "#":r"\#", "_":r"\_", "{":r"\{",
             "}":r"\}", "~":r"\textasciitilde{}"}
    for k, v in chars.items():
        text = text.replace(k, v)
    return text
```

OUTPUT

str : Texte sécurisé LaTeX — tous les caractères spéciaux neutralisés

Pourquoi ? En LaTeX, % commence un commentaire. Sans cette fonction, *"la TVA est de 20%"* supprimerait le reste de la ligne dans la compilation !

Le Tableau de Bord : agent_config.json

Principe

Un seul fichier contrôle **tout le système**.
Changer le sujet = modifier 2 lignes.

```
{
  "project_title": "SUJET ICI",
  "theme_description": "La question...",
  "author_name": "Bilel Ammouri",
  "article_count": 50,
  "review_word_count": 3000,
  "scopus_api_key": "...",
  "gemini_api_key": "...",
  "model_name": "gemini-3.1-flash"
}
```

Param.	Rôle
project_title	Titre du rapport
theme	Requête IA
article_count	Volume collecte
review_word_count	Longueur texte
gemini_api_key	Accès IA

Lancement du Système

Installation

```
pip install -r requirements.txt
```

Lancement complet

```
python master_agent.py
# Choisir source [1-3]
```

Tests individuels

```
python core/scopus_collector.py
python core/paper_ranker.py
python core/review_drafter.py
```

Fichiers produits

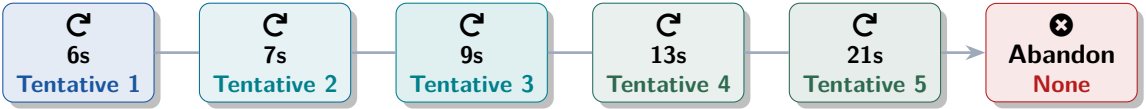
- scopus_metadata.csv
- openalex_metadata.csv
- collected_metadata.csv
- results.bib
- citation_metrics.csv
- bibliometric_summary.png
- ranked_articles.json
- main.tex
- main.pdf ✔

Le Système Ne Plante Jamais

Scénario d'erreur	Comportement	Solution automatique
Scopus API échoue	Warning	Utilise OpenAlex seul
Gemini rate limit	Attente auto	Retry $\times 5$ (backoff exponentiel)
R non installé	Warning	Classement sans métriques R
Résumé absent	"" vide	`reconstruct_abstract()` $\rightarrow$ ""
JSON malformé	Repli	Structure 4 sections par défaut
LaTeX échoue	Warning	.tex conservé pour compilation manuelle

ARA est conçu pour toujours produire un résultat, même partiel, plutôt que de crasher.

Exponential Backoff — Gerer les Limites d'API



Formule du delai

$$\text{delai} = 2^{\text{tentative}} + \text{aleatoire} + 5 \text{ secondes}$$

L'aleatoire evite que plusieurs processus paralleles reeessayent exactement au meme moment.

Synthèse : Les 18 Fonctions

Fonction	Agent	Input	Output	Rôle
`find_r_executable()`	Master	—	str/None	Localise Rscript
`run_step()`	Master	name, cmd	bool	Exécute 1 étape
`merge_results()`	Master	sources	None	Fusionne CSVs
`main()`	Master	—	None	Orchestre tout
`get_config()`	Scopus/OA	—	dict	Charge config
`suggest_boolean_query()`	Scopus	client,theme	str	Requête Scopus IA
`fetch_scopus_data()`	Scopus	query,key	list	Appel API
`process_entries()`	Scopus	entries	DataFrame	Normalise
`export_to_bibtex()`	Scopus/OA	df	None	Export .bib
`suggest_search_query()`	OpenAlex	client,theme	str	Requête OA IA
`reconstruct_abstract()`	OpenAlex	inv_index	str	Décode résumé
`fetch_openalex_data()`	OpenAlex	query,key	list	Appel API OA
`process_results()`	OpenAlex	results	DataFrame	Normalise OA
`rank_papers()`	Ranker	—	None $\rightarrow$ JSON	Score + Top 15
`load_data()`	Drafter	—	list	Charge articles
`latex_escape()`	Drafter	str	str	Sécurise LaTeX
`call_with_retry()`	Drafter	prompt	str	Gemini + retry
`draft_section()`	Drafter	context	str	Rédige section

Feuille de Route Recommandée

Court terme (1–3 mois)

- Ajouter PubMed, ArXiv comme sources
- Interface web (Streamlit) pour non-techniciens
- Rapport en français ou arabe

Moyen terme (3–6 mois)

- Veilles hebdomadaires automatisées
- Connexion Power BI pour dashboards
- Historique des résultats en base SQL

Long terme (>6 mois)

- Parallélisation (collecte simultanée)
- Fine-tuning du modèle sur données métier
- API interne pour intégration SIRH/ERP
- Scoring sémantique (embeddings)

Ce Qu'il Faut Retenir — Les 6 Points Clés

- **Gain $\times$1000** : 3 semaines $\rightarrow$ 15 minutes automatisées
- **6 agents spécialisés** travaillant en séquence intelligente
- **18 fonctions Python** documentées et testables individuellement
- **Résilience** : aucune panne n'arrête complètement le système
- **Livrable professionnel** : rapport PDF académique prêt à publier
- **Configuration triviale** : 1 fichier JSON pour tout contrôler

ARA — La recherche académique à la vitesse de l'Intelligence Artificielle

Merci de votre attention

Bilel Ammouri

Autonomous Research Agent — Version 2.0

Avril 2025

Thème analysé :

"Équité intergénérationnelle dans les politiques de décarbonation et structures fiscales"